MODULE 4 – MODERN C# & .NET

1. Introduction to Modern C#

Modern C# is an evolving programming language that provides features for developing secure, maintainable, high-performance applications.

Modern C# focuses on:

- Writing clean and readable code.
- Improving application performance.
- Reducing unnecessary code.
- Supporting asynchronous programming.
- Providing better type safety.
- Supporting modern application development.

Important modern C# features include:

- Nullable reference types.
- Records.
- Pattern matching.
- Lambda expressions.
- Extension methods.
- Generics.
- Async and await.
- Collection expressions.
- Global using directives.
- File-scoped namespaces.
- Primary constructors.

2. Difference Between .NET Framework and Modern .NET

.NET Framework:

- Original Windows-focused .NET implementation.
- Mainly used for traditional Windows applications.
- Primarily supports Windows.

Modern .NET:

- Cross-platform development platform.
- Supports Windows, Linux and macOS.
- Used for web APIs, cloud applications, desktop applications and services.

- Provides modern performance and development features.

Modern .NET is the continuation of the .NET Core platform.

3. .NET SDK, Runtime and CLI

.NET SDK (Software Development Kit):

The SDK provides the tools required to create, build, test and publish .NET applications.

It includes:

- .NET CLI.
- Compilers.
- Build tools.
- Templates.
- Required development components.

.NET Runtime:

The runtime provides the environment required to execute .NET applications.

.NET CLI (Command Line Interface):

The CLI allows developers to perform .NET development tasks using commands.

Important commands:

```C#
dotnet --version
dotnet --info
dotnet new console
dotnet new webapi
dotnet new mvc
dotnet restore
dotnet build
dotnet run
dotnet test
dotnet publish
```

Explanation:

- dotnet --version: Displays the installed SDK version.

- dotnet --info: Displays detailed .NET information.
- dotnet new: Creates a new project.
- dotnet restore: Restores project dependencies.
- dotnet build: Builds the project.
- dotnet run: Builds and runs the application.
- dotnet test: Executes tests.
- dotnet publish: Prepares the application for deployment.

4. Modern C# Project Structure

A typical modern .NET project contains:

```
C#
MyApplication/
│
├── bin/
├── obj/
├── Properties/
├── Program.cs
├── MyApplication.csproj
├── appsettings.json
└── appsettings.Development.json
```

Explanation:

- bin: Contains compiled application output.
- obj: Contains intermediate build files.
- Properties: Contains project-related settings.
- Program.cs: Entry point of the application.
- .csproj: Project configuration file.
- appsettings.json: Stores application configuration.
- appsettings.Development.json: Stores development-specific configuration.

5. Understanding the .csproj File

The .csproj file contains project configuration and dependency information.

Example:

```
C#
<Project Sdk="Microsoft.NET.Sdk.Web">
```

```
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

</Project>
```

Explanation:

- Project Sdk: Specifies the SDK used by the project.
- TargetFramework: Specifies the target .NET framework.
- Nullable: Enables nullable reference type analysis.
- ImplicitUsings: Automatically includes common using directives.

6. File-Scoped Namespaces

A file-scoped namespace allows developers to declare a namespace without adding an extra block of braces.

Traditional namespace:

```
C#
namespace MyApplication
{
    class Student
    {
    }
}
```

File-scoped namespace:

```
C#
namespace MyApplication;

class Student
{
```

```
}
```

Advantages:

- Reduces unnecessary indentation.
- Makes code easier to read.
- Reduces additional braces.

7. Global Using Directives

Global using allows a namespace to be imported across all files in a project.

Example:

```C#
global using System;
global using System.Collections.Generic;
global using System.Linq;
```

After declaring global using directives, other files in the project can use these namespaces without importing them separately.

8. Implicit Usings

Implicit usings automatically include commonly used namespaces based on the project SDK.

Example:

```C#
<ImplicitUsings>enable</ImplicitUsings>
```

When implicit usings are enabled, developers do not need to manually write every common using statement.

9. Nullable Reference Types

Nullable reference types help developers identify possible null reference errors.

A reference variable normally indicates that it should contain a non-null value when nullable reference types are enabled.

Example:

```C#
string name = "Devadharshini";

string? address = null;
```

Explanation:

- string: Indicates a non-null reference is expected.
- string?: Indicates that null is also allowed.

Null Check:

```C#
if (address != null)
{
    Console.WriteLine(address.Length);
}
```

Null-Coalescing Operator (??):

Returns the right-hand value when the left-hand value is null.

```C#
string result = address ?? "Not Available";
```

Null-Conditional Operator (?.):

Accesses a member only when the object is not null.

```C#
int? length = address?.Length;
```

Null-Coalescing Assignment (??=):

Assigns a value only when the variable is null.

```C#
string? city = null;

city ??= "Chennai";
```

10. Records in C#

A record is a reference type designed mainly for storing data.

Records provide value-based equality by default.

Example:

```C#
public record Student(int Id, string Name);

Student student1 = new Student(1, "Devadharshini");
Student student2 = new Student(1, "Devadharshini");

Console.WriteLine(student1 == student2);
```

Output:

```C#
True
```

In ordinary classes, equality generally compares object references unless equality is overridden.

Advantages of Records:

- Less boilerplate code.
- Value-based equality.
- Useful for data transfer.
- Supports non-destructive modification.

Using with Expressions:

```C#
Student student1 = new Student(1, "Devadharshini");
```

```
Student student2 = student1 with { Name = "Divya" };
```

The with expression creates a new record with the specified change.

11. Init-Only Properties

The init accessor allows a property to be assigned during object initialization.

Example:

```
C#
class Student
{
    public int Id { get; init; }

    public string Name { get; init; }
}
```

Object initialization:

```
C#
Student student = new Student
{
    Id = 1,
    Name = "Devadharshini"
};
```

The init-only property cannot normally be assigned again after initialization.

12. Required Members

The required keyword indicates that a property or field must be initialized when an object is created.

Example:

```C#
class Employee
{
    public required int Id { get; set; }

    public required string Name { get; set; }
}
```

Object creation:

```C#
Employee employee = new Employee
{
    Id = 101,
    Name = "Divya"
};
```

Required members help prevent objects from being created without essential data.

13. Pattern Matching

Pattern matching allows developers to check values against different patterns.

It makes conditional logic more readable.

Type Pattern:

```C#
object value = "Hello";

if (value is string)
{
    Console.WriteLine("Value is a string");
}
```

Type Pattern with Variable:

```C#
if (value is string text)
{
    Console.WriteLine(text.Length);
}
```

Relational Pattern:

```C#
int mark = 85;

if (mark is >= 75 and <= 100)
{
    Console.WriteLine("Distinction");
}
```

Logical Patterns:

- and: Both conditions must match.
- or: At least one condition must match.
- not: Negates a pattern.

Example:

```C#
int age = 25;

if (age is >= 18 and < 60)
{
    Console.WriteLine("Eligible");
}
```

14. Switch Expressions

A switch expression provides a concise way to return values based on conditions.

Traditional switch statement:

```C#
int day = 1;

string result;

switch (day)
{
    case 1:
        result = "Monday";
        break;

    case 2:
        result = "Tuesday";
        break;

    default:
        result = "Invalid";
        break;
}
```

Modern switch expression:

```C#
int day = 1;

string result = day switch
{
    1 => "Monday",
    2 => "Tuesday",
    3 => "Wednesday",
    _ => "Invalid"
};
```

The underscore (_) represents the default case.

15. Tuples

A tuple allows multiple values to be grouped into a single value.

Example:

```
C#
var student = (Id: 101, Name: "Devadharshini", Mark: 90);

Console.WriteLine(student.Id);
Console.WriteLine(student.Name);
Console.WriteLine(student.Mark);
```

Returning Multiple Values Using Tuples:

```
C#
(int, string) GetStudent()
{
    return (101, "Devadharshini");
}
```

Calling the method:

```
C#
var student = GetStudent();

Console.WriteLine(student.Item1);
Console.WriteLine(student.Item2);
```

Named tuples allow values to be accessed using meaningful names instead of Item1 and Item2.

16. Deconstruction

Deconstruction allows tuple or object values to be assigned to separate variables.

Example:

```
C#
var student = (101, "Divya");

var (id, name) = student;
```

```
Console.WriteLine(id);
Console.WriteLine(name);
```

It reduces the need to access each tuple element separately.

17. Target-Typed New Expressions

The target-typed new expression allows the compiler to determine the object type from the declared variable type.

Traditional syntax:

```C#
List<int> numbers = new List<int>();
```

Modern syntax:

```C#
List<int> numbers = new();
```

This reduces repetitive code.

18. Collection Expressions

Collection expressions provide a concise syntax for creating collections.

Example:

```C#
int[] numbers = [10, 20, 30, 40];

List<string> names = ["Arun", "Divya", "Kumar"];
```

The spread element can be used to include elements from another collection.

```C#
int[] first = [1, 2, 3];
```

```
int[] second = [.. first, 4, 5];
```

Collection expressions are supported in modern C# versions. Their use depends on the target type and compiler version.

19. Index and Range Operators

Index and range operators provide convenient ways to access elements from arrays and other supported types.

Index Operator (^):

The ^ operator counts from the end of a collection.

```
C#
int[] numbers = [10, 20, 30, 40, 50];

Console.WriteLine(numbers[^1]);
```

Output:

```
C#
50
```

Range Operator (..):

The range operator selects a portion of a collection.

```
C#
int[] numbers = [10, 20, 30, 40, 50];

int[] result = numbers[1..4];
```

Output:

```
None
20, 30, 40
```

20. Extension Methods

Extension methods allow developers to add methods to existing types without modifying their original source code or creating a derived type.

Example:

```
C#
public static class StringExtensions
{
    public static string ToUpperCase(this string value)
    {
        return value.ToUpper();
    }
}
```

Usage:

```
C#
string name = "devadharhsini";

Console.WriteLine(name.ToUpperCase());
```

Important points:

- Extension methods are defined inside static classes.
- Extension methods must be static.
- The first parameter uses the this keyword.
- They are called like instance methods.

21. Generics

Generics allow developers to create reusable classes, methods and interfaces that work with different data types.

Example:

```C#
class Box<T>
{
    public T Value { get; set; }

    public Box(T value)
    {
        Value = value;
    }
}
```

Usage:

```C#
Box<int> numberBox = new Box<int>(100);

Box<string> textBox = new Box<string>("Hello");
```

Advantages:

- Code reusability.
- Type safety.
- Reduces unnecessary type conversions.
- Improves maintainability.

22. Generic Constraints

Generic constraints restrict the types that can be used with a generic type or method.

Example:

```C#
class Repository<T> where T : class
{
    public T? Data { get; set; }
}
```

Common constraints:

- where T : class – T must be a reference type.
- where T : struct – T must be a non-nullable value type.
- where T : new() – T must have a public parameterless constructor.
- where T : notnull – T must be a non-nullable type.
- where T : SomeBaseClass – T must inherit from a specified base class.

23. IEnumerable and IQueryable

Both IEnumerable and IQueryable are used to work with sequences of data.

IEnumerable:

- Represents a sequence of elements.
- Commonly used for in-memory collections.
- Queries are generally executed locally.

Example:

```
C#
IEnumerable<int> numbers = new List<int>
{
    10, 20, 30, 40
};

var result = numbers.Where(n => n > 20);
```

IQueryable:

- Represents a query that can be translated by a query provider.
- Commonly used with Entity Framework Core.
- Allows filtering and projection to be translated into database queries when supported.

Example:

```
C#
IQueryable<Student> students = dbContext.Students;

var result = students.Where(s => s.Mark > 80);
```

Important difference:

IEnumerable generally works with in-memory iteration, whereas IQueryable allows a provider to translate query expressions, such as into SQL.

24. Async and Await in Modern C#

Asynchronous programming allows an application to perform operations without blocking the calling thread while waiting for an asynchronous operation.

It is especially useful for:

- Database operations.
- API calls.
- File operations.
- Network operations.

Example:

```
C#
public async Task<string> GetDataAsync()
{
    await Task.Delay(2000);

    return "Data Received";
}
```

Calling the method:

```
C#
string result = await GetDataAsync();

Console.WriteLine(result);
```

Task:

Represents an asynchronous operation that does not return a value.

Task<T>:

Represents an asynchronous operation that returns a value of type T.

ValueTask<T>:

A value-type alternative that can help reduce allocations in some performance-sensitive asynchronous scenarios.

25. CancellationToken

CancellationToken is used to request cancellation of an operation.

It allows long-running operations to stop when they are no longer needed.

Example:

```
C#
public async Task ProcessAsync(CancellationToken cancellationToken)
{
    for (int i = 0; i < 5; i++)
    {
        cancellationToken.ThrowIfCancellationRequested();

        await Task.Delay(1000, cancellationToken);

        Console.WriteLine($"Processing {i}");
    }
}
```

Important points:

- CancellationToken does not forcibly terminate code.
- It communicates a cancellation request.
- The executing method must observe and respond to the request.

26. IDisposable and Using Statement

IDisposable is an interface used to release resources such as file handles, database connections and streams.

Example:

```
C#
using (StreamWriter writer = new StreamWriter("sample.txt"))
{
    writer.WriteLine("Hello .NET");
```

```
}
```

The using statement ensures that the object is disposed when execution leaves the block, including when an exception occurs.

Using Declaration:

```
C#
using StreamWriter writer = new StreamWriter("sample.txt");

writer.WriteLine("Hello .NET");
```

The object is disposed at the end of its containing scope.

IAsyncDisposable:

Used for asynchronous resource cleanup through DisposeAsync().

27. Garbage Collection

Garbage Collection (GC) is the automatic memory management feature of .NET.

It identifies objects that are no longer reachable and reclaims their memory.

Important concepts:

- Managed Heap: Memory area where reference-type objects are generally allocated.
- Garbage Collector: Reclaims memory occupied by unreachable managed objects.
- Generation 0: Stores newer and short-lived objects.
- Generation 1: Acts as an intermediate generation.
- Generation 2: Stores longer-lived objects.

Advantages:

- Reduces manual memory management.
- Helps prevent many memory-related errors.
- Automatically reclaims eligible managed objects.

Important: Garbage Collection does not replace the need to dispose resources such as database connections and file handles.

28. .NET Application Hosting

Hosting provides the environment required to configure and run a .NET application.

The Generic Host is commonly used in modern .NET applications.

It provides:

- Dependency Injection.
- Configuration.
- Logging.
- Application lifetime management.
- Hosted services.

Example:

```
C#
var builder = Host.CreateApplicationBuilder(args);

builder.Services.AddSingleton<MyService>();

var host = builder.Build();

host.Run();
```

29. Dependency Injection in Modern .NET

Dependency Injection (DI) is a design pattern used to provide required dependencies to a class instead of creating them manually.

Example:

```
C#
public interface IMessageService
{
    void SendMessage();
}

public class EmailService : IMessageService
{
```

```
    public void SendMessage()
    {
        Console.WriteLine("Email Sent");
    }
}
```

Registering the service:

```
C#
builder.Services.AddScoped<IMessageService, EmailService>();
```

Using constructor injection:

```
C#
public class HomeController : Controller
{
    private readonly IMessageService _messageService;

    public HomeController(IMessageService messageService)
    {
        _messageService = messageService;
    }
}
```

The DI container creates and provides the required service.

DI Service Lifetimes:

1. Transient:
    - Creates a new service instance each time it is requested from the container.
2. Scoped:
    - Creates one service instance per scope.
    - In a typical ASP.NET Core web application, this is usually one instance per HTTP request.
3. Singleton:
    - Creates one service instance for the lifetime of the service provider.

30. Configuration in .NET

Configuration is used to store application settings separately from application code.

Common configuration sources:

- appsettings.json
- appsettings.Development.json
- Environment variables
- Command-line arguments
- User Secrets
- External configuration providers

Example appsettings.json:

```
C#
{
  "AppSettings": {
    "ApplicationName": "StudentManagement",
    "Version": "1.0"
  }
}
```

Reading configuration:

```
C#
string? appName =
    builder.Configuration["AppSettings:ApplicationName"];
```

31. Environment-Specific Configuration

Different environments may require different configurations.

Common environments:

- Development
- Staging
- Production

Example:

appsettings.Development.json:

```C#
{
  "AppSettings": {
    "ApplicationName": "StudentManagement-Development"
  }
}
```

appsettings.Production.json:

```C#
{
  "AppSettings": {
    "ApplicationName": "StudentManagement-Production"
  }
}
```

The application environment determines which environment-specific settings are loaded.

32. Options Pattern

The Options Pattern is used to bind configuration values to strongly typed C# classes.

Example appsettings.json:

```C#
{
  "EmailSettings": {
    "Host": "smtp.example.com",
    "Port": 587
  }
}
```

Creating a settings class:

```C#
public class EmailSettings
{
```

```
    public string Host { get; set; } = string.Empty;

    public int Port { get; set; }
}
```

Registering the settings:

```
C#
builder.Services.Configure<EmailSettings>(
    builder.Configuration.GetSection("EmailSettings"));
```

Injecting IOptions:

```
C#
public class EmailService
{
    private readonly EmailSettings _settings;

    public EmailService(IOptions<EmailSettings> options)
    {
        _settings = options.Value;
    }
}
```

Common options interfaces:

- IOptions<T>: Provides configured values.
- IOptionsSnapshot<T>: Provides scoped options and supports reload behavior when supported by the configuration source.
- IOptionsMonitor<T>: Supports retrieving current values and change notifications when supported.

33. Logging in .NET

Logging is used to record information about an application's execution.

It helps developers:

- Identify errors.
- Monitor application behavior.
- Debug problems.
- Track important operations.

Example:

```C#
public class StudentService
{
    private readonly ILogger<StudentService> _logger;

    public StudentService(ILogger<StudentService> logger)
    {
        _logger = logger;
    }

    public void Display()
    {
        _logger.LogInformation("Student service executed");

        _logger.LogError("An error occurred");
    }
}
```

Important logging levels:

- Trace
- Debug
- Information
- Warning
- Error
- Critical

34. Middleware in ASP.NET Core

Middleware is software that is assembled into the HTTP request pipeline.

Each middleware component can:

- Process an incoming HTTP request.
- Pass the request to the next middleware.

- Process the outgoing HTTP response.
- Stop the pipeline by not calling the next middleware.

Example:

```
C#
app.Use(async (context, next) =>
{
    Console.WriteLine("Before Middleware");

    await next();

    Console.WriteLine("After Middleware");
});
```

Important middleware methods:

- Use(): Adds middleware that can call the next component.
- Run(): Adds terminal middleware that ends the pipeline.
- Map(): Branches the pipeline based on a request path.

Middleware execution order is important because it affects request processing.

35. Hosted Services and Background Tasks

Hosted services allow background tasks to run as part of a .NET application.

IHostedService provides StartAsync() and StopAsync() methods.

Example:

```
C#
public class MyBackgroundService : BackgroundService
{
    protected override async Task ExecuteAsync(
        CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            Console.WriteLine("Background task running");
```

```
            await Task.Delay(5000, stoppingToken);
        }
    }
}
```

Registering the service:

```
C#
builder.Services.AddHostedService<MyBackgroundService>();
```

Common uses:

- Sending scheduled emails.
- Processing queued tasks.
- Running periodic background operations.
- Monitoring application activities.

36. HttpClient and IHttpClientFactory

HttpClient is used to send HTTP requests to external services and APIs.

Example:

```
C#
using HttpClient client = new HttpClient();

string response = await client.GetStringAsync(
    "https://example.com");

Console.WriteLine(response);
```

In ASP.NET Core, IHttpClientFactory is recommended for creating and managing HttpClient instances.

Registration:

```
C#
builder.Services.AddHttpClient();
```

Usage:

```
C#
public class ApiService
{
    private readonly HttpClient _client;

    public ApiService(HttpClient client)
    {
        _client = client;
    }

    public async Task<string> GetDataAsync()
    {
        return await _client.GetStringAsync(
            "https://example.com");
    }
}
```

Typed client registration:

```
C#
builder.Services.AddHttpClient<ApiService>();
```

37. NuGet Package Manager

NuGet is the package management system for .NET.

It allows developers to install and manage external libraries and dependencies.

Examples:

- Microsoft.EntityFrameworkCore
- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.AspNetCore.Authentication.JwtBearer
- Swashbuckle.AspNetCore

- xUnit

Important CLI commands:

```
C#
dotnet add package PackageName
dotnet remove package PackageName
dotnet list package
dotnet restore
```

NuGet packages help developers reuse existing functionality instead of writing everything from scratch.

38. Unit Testing in .NET

Unit testing verifies individual units of code, such as methods or classes.

Common testing frameworks:

- xUnit
- NUnit
- MSTest

Example using xUnit:

```
C#
public class Calculator
{
    public int Add(int a, int b)
    {
        return a + b;
    }
}
```

Test class:

```
C#
public class CalculatorTests
{
    [Fact]
```

```
    public void Add_ReturnsCorrectResult()
    {
        Calculator calculator = new Calculator();

        int result = calculator.Add(10, 20);

        Assert.Equal(30, result);
    }
}
```

Important xUnit attributes:

- [Fact]: Defines a test without input parameters.
- [Theory]: Defines a parameterized test.
- [InlineData]: Supplies test input values.

Example:

```
C#
[Theory]
[InlineData(10, 20, 30)]
[InlineData(5, 5, 10)]
public void Add_ReturnsExpectedResult(
    int a, int b, int expected)
{
    Calculator calculator = new Calculator();

    int result = calculator.Add(a, b);

    Assert.Equal(expected, result);
}
```

39. Application Publishing and Deployment

Publishing prepares an application for deployment.

Command:

```C#
dotnet publish -c Release
```

Important terms:

Build:

- Compiles the application.
- Produces build output.

Publish:

- Prepares the application and required files for deployment.

Framework-Dependent Deployment:

- The target machine must have the required .NET runtime installed.

Self-Contained Deployment:

- Includes the required .NET runtime with the published application.

Release Configuration:

- Used to generate optimized build output for deployment.